

PhonoScript

Language and Engine Manual

Optimality Theory · Harmonic Grammar · Maximum Entropy
Serial evaluation · Second-Order comparison · Q-Calculus

Alexandre Menezes Barroso

alexandrebarroso.com

Language version 3 · Component version 1.1.0 · 2026

MIT License

Contents

1	Purpose and architecture	1
2	Installation and command-line use	1
2.1	Complete command-line reference	2
2.2	Rust embedding surface	3
3	Source and execution model	3
4	Values, expressions, and control flow	4
5	Complete lexical reference	5
6	Complete grammar	6
6.1	Operator semantics and precedence	7
6.2	Bindings, scopes, and functions	8
6.3	Gradual static analysis	8
7	Modules and reusable analysis libraries	9
8	Diagnostic catalogue	9
9	Projects, tableaux, and explicit violation ledgers	11
10	A complete finite script	12
11	Evaluation, learning, and serial analysis	12
12	Second-Order comparison and Q-Calculus	13
13	Structured phonology and bounded GEN	14
13.1	Structured-form literal reference	15
13.2	Finite-GEN specification reference	15
13.3	Complete structured-candidate records	16
14	Complete builtin reference	17
14.1	General values and assertions	17
14.2	Project and tableau mutation	17
14.3	Constraints, candidates, and violation cells	18
14.4	Evaluation and learning	19
14.5	Serial, Second-Order, and Q operations	20

14.6 Text generation and structured phonology	21
14.7 Files and publication export	21
15 Scripting patterns for reproducible analyses	22
15.1 Build, calculate, and assert	22
15.2 Status-first handling of typed results	22
15.3 Exact and approximate assertions	23
15.4 Reusable modules	23
15.5 Multi-tableau projects	23
16 The .ottab document format	24
17 File effects and export	24
18 Implemented capabilities and boundaries	24
19 Validation and source-replication scope	25
19.1 Source-exact finite cases and exact ledger transcription	26
19.2 Bounded reconstructions	26
20 Performance and scientific limit	27
21 Security, contribution discipline, and license	28

1 Purpose and architecture

PhonoScript is a standalone UTF-8 scripting language and analysis engine for finite constraint-based phonology. *PhonoScript* is the language name; `.phont` is its source-file extension. The command-line interpreter can run without a graphical application. ConvalGEN depends on the same crate for language execution, project validation, evaluator mathematics, typed Second-Order comparison, Q-Calculus, and vector export; PhonoScript does not depend on ConvalGEN.

The implemented evaluator families are strict Optimality Theory (OT), Harmonic Grammar (HG), finite Maximum Entropy grammar (MaxEnt), and finite serial OT/HG. The same engine also implements consequence-relative Second-Order comparison and Q-Calculus operations. The engine acts on a declared finite support. It does not claim that a user's GEN is linguistically complete or that every formalism in phonology reduces to the implemented contract.

Three version numbers are independent:

- Project PhonoScript component version: 1.1.0;
- PhonoScript language version: 3; and
- `.ottab` document schema version: 4.

An unknown operation or document version is refused rather than interpreted through an implicit compatibility rule.

2 Installation and command-line use

Rust 1.88 or newer is required for a source installation. From the Project PhonoScript repository root:

```
cargo install --locked --path phonoscript --bin phonoscript
```

Run or check a script:

```
phonoscript analysis.phont
phonoscript --check analysis.phont
phonoscript --json analysis.phont
phonoscript --module-root analyses analyses/main.phont
```

Run directly from the workspace without installation:

```
cargo run --release -p phonoscript -- analysis.phont
```

The packaged macOS and Linux archives include `install.sh`. From the unpacked directory, the default installs for the current user beneath `$HOME/.local`; a system or explicit-prefix installation is selected deliberately:

```
./install.sh --user
sudo ./install.sh --system
./install.sh --prefix /chosen/prefix
```

Set `PHONOSCRIPT_PREFIX` to change the default prefix without a flag. The installer copies the interpreter, license, PDF documentation, validation programs, and fixtures, then starts `phonoscript --version`. Add the selected `bin` directory to `PATH` if the installer reports that it is absent.

The Windows archive includes a current-user PowerShell installer:

```
powershell -ExecutionPolicy Bypass -File .\install-user.ps1
```

Its default destination is %LOCALAPPDATA%\Programs\PhonoScript. It adds that directory to the user PATH, registers the executable in the current-user application paths, and associates .phont with the interpreter. -InstallRoot selects another current-user directory.

The interpreter accepts --base to begin from a document and --write to save the validated result. --emit serializes a current .ottab project as a self-contained PhonoScript payload. --json emits diagnostics, exactness-boundary records, run statistics, the resulting document, and the final value as one JSON record. The final value is structural: exact numbers carry explicit numerator and denominator strings, approximate numbers carry an explicit approximate tag and binary floating-point value, and lists and records preserve that distinction recursively.

The checked-in standalone archives include a locally exercised Apple-silicon macOS build. It is ad-hoc signed for local integrity and is not Developer-ID signed or notarized for public Gatekeeper distribution. Linux and Windows archives produced by the macOS cross-build gate are linked for their declared targets and inspected for binary format and path hygiene, but they are not native execution evidence. The matching release-workflow jobs remain the runtime gate on Linux and Windows.

2.1 Complete command-line reference

The interpreter has three invocation forms:

```
phonoscript [OPTIONS] SCRIPT.phont [BASE.ottab]
phonoscript [OPTIONS] -
phonoscript --emit PROJECT.ottab [--write SCRIPT.phont]
```

The optional positional BASE.ottab is equivalent to --base BASE.ottab. A single hyphen reads UTF-8 source from standard input; imports are unavailable there because the stream has no canonical module location.

Option	Meaning and constraints
--base PATH	Begin with a validated self-contained .ottab project instead of a blank project.
--module-root PATH	Resolve a file entry and every selective import beneath this canonical root; unavailable with standard input or --emit.
--write PATH	After successful execution, atomically save the resulting validated project as .ottab; with --emit, write emitted .phont source instead.
--emit PATH	Decode a validated .ottab project and render a self-contained PhonoScript reconstruction; does not accept a script input.
--check	Lex, parse, resolve modules, and run static analysis without executing; mutually exclusive with --emit and --write.
--json	Emit exactly one machine-readable JSON report; mutually exclusive with --quiet.
--quiet	Suppress successful text output; diagnostics still use standard error.
-h, --help	Print the invocation reference and exit successfully.
-V, --version	Print component and language versions and exit successfully.

Exit status is zero only when the requested check, run, or emission succeeds. Argument errors, decoding errors, check errors, runtime errors, failed assertions, validation failures, and failed file effects exit one. Warnings such as an explicit exact-to-floating boundary do not by themselves make the run fail.

Two accepted flag combinations have deliberately documented current behavior: `--emit --json` still emits ordinary PhonoScript text, so the JSON flag has no effect in emission mode; `--check --base PATH` checks source but does not decode or consult the base project. Scripts and automation should not use either inert flag as evidence that JSON emission or base-document validation occurred.

With `--check --json`, the report fields are `status`, `language`, `language_version`, `source`, and `diagnostics`. A full `--json` execution adds `selected_tableau`, captured output, structural value, `rendered_value`, `approximate_boundaries`, `statistics`, and the complete resulting document. Exact numbers are encoded with the tag `exact_number` plus numerator and denominator strings; approximate numbers carry the tag `approximate_number` and a binary floating-point value. Lists, records, Booleans, text, and null retain distinct tagged forms recursively.

Each JSON diagnostic retains `source`, stable code, severity, message, primary start/end positions, related spans, help text when available, and source-aware call frames. The statistics object reports modules loaded, steps, statements, expressions, calls, loop iterations, engine calls, exact-to-engine conversions, and queued file effects. Automation should inspect `status` and the process exit code; it should not infer success from an empty captured-output array.

2.2 Rust embedding surface

Applications embedding the crate may call `run` or `run_named` for in-memory source and an initial document, and `check` or `check_named` for frontend/static diagnostics. `run_file` and `check_file` add canonical module-root resolution. The corresponding `_with_limits` variants accept an explicit `RuntimeLimits` value. `try_emit` returns a checked self-contained reconstruction, while `emit` is its convenience form.

`RunResult` carries the private-or-committed document, final value, captured standard output, diagnostics, approximation-boundary records, statistics, selected-tableau coordinate, and commit state. Embedders should test the result's success/commit contract and retain diagnostics; extracting the returned document from a failed run does not recover partial mutations. The file-running functions confine imports, not save/export destinations, so a host requiring stronger filesystem isolation must impose it outside the language runtime.

3 Source and execution model

Source is UTF-8. Identifiers accept Unicode letters, numbers after the first character, underscores, and combining marks. Newlines or semicolons terminate statements; newlines inside parentheses, brackets, braces, calls, lists, and records are soft separators. `#` and `//` begin line comments. Block comments written `/* . . . */` may nest. A Unix `#!/usr/bin/env phonoscript` line is valid trivia.

Strings may use single or double quotes. They support backslash, quote, newline, carriage-return, tab, null, and Unicode scalar escapes. Record fields may use identifier keys or quoted keys. Member access with a dot and bracket access are equivalent for identifier-shaped keys; bracket access also supports keys containing spaces.

Execution is deterministic and transactional. The runtime works on a clone of the input project. It commits project mutations and queued file effects only after parsing, static analysis, execution, normalization, and document validation all succeed. A failure therefore leaves the original project unchanged.

Language version 3 admits selective imports from local `.phont` modules. An entry file imports only names that another file explicitly exports; aliases are lexical, immutable bindings. Resolution is relative to the importing file and is confined to an explicit canonical module root. Absolute paths, non-`.phont` files, paths or symlinks escaping the root, dependency cycles, missing exports, excessive depth, and ex-

cessive graph size are refused with source-located diagnostics. Remote loading and a package registry are outside the language contract. `--base` loads project data; it does not include executable declarations.

```
// measures.phont
export let source = "Kager 1999, p. 16"
export fn build() {
  tableau_new("final voicing", "/bɛd/")
  tableau_evaluator("OT")
  // add the explicitly transcribed constraints, candidates, and marks
}

// main.phont
import { source, build as build_analysis } from "./measures.phont"
print(source)
build_analysis()
evaluate()
```

Imported modules are declaration-only. At their top level they may contain imports, function declarations, and immutable call-free and assignment-free `let` initializers. Printing, assertions, project mutation, and file effects belong inside an exported function and occur only when the entry file calls that function explicitly. The complete graph shares one transaction: any diagnostic discards project changes and queued writes from every module. The default graph ceiling is 256 distinct files, import depth 64, and 16 MiB of aggregate UTF-8 source. The API exposes deliberate overrides; the CLI uses the production defaults.

Diagnostics use stable prefixes: PSL for lexical errors, PSP for parser errors, PSA for static-analysis errors, and PSR for runtime errors. Command-line diagnostics retain the source filename, a primary span, related spans when the frontend can identify them, and source-aware function-call frames for runtime faults. Locations use one-based lines and Unicode-scalar columns. The static analyzer checks lexical resolution, same-scope duplicates, immutable assignment, return placement, statically known arity and type errors, and some literal-domain errors. Candidate lookup, matrix width, evaluator admission, and project-state requirements remain runtime or engine checks.

Production execution is bounded to 1,000,000 steps, 100,000 loop iterations, call depth 256, 250,000 elements per collection, 1,000,000 bytes of exact-number storage, and 4,000,000 bytes of printed output. Finite GEN separately checks its declared candidate, expansion, variant, segment, and derivation bounds against those ceilings.

4 Values, expressions, and control flow

Values are exact or approximate numbers, booleans, text, lists, records, and `null`. Integer, rational, decimal, and scientific source literals are exact arbitrary-precision rationals. Arithmetic over exact operands stays exact. Numerical engine results render with a leading tilde; an approximate center is not equal to an exact rational merely because their displayed digits coincide.

```
let third = 1/3
let analysis = {
  name: "final devoicing",
  "source page": 16,
  optional_note: null,
}
```

```
assert_equal(third + 2/3, 1)
assert_equal(analysis["source page"], 16)
```

Expressions support arithmetic, remainder, comparison, equality, and boolean and, or, and not; symbolic forms &&, ||, and ! are also accepted. Assignment is allowed only to a var binding. Functions are lexical closures. Nested scopes may shadow an outer binding, while same-scope duplication is an error.

```
var total = 0

fn triangular(n) {
  if n <= 0 { return 0 }
  return n + triangular(n - 1)
}

for n in range(1, 6) {
  total = total + n
}

while total < 20 {
  total = total + 1
}

assert_equal(triangular(5), 15)
```

for iterates over a list or over the Unicode scalars in text. range(end), range(start,end), and range(start,end,step) use an exclusive end. Zero step and a range larger than the collection ceiling are refused.

General builtins are:

```
print(values...)
assert(condition[, message])
assert_equal(left, right[, message])
assert_approx(left, right, tolerance[, message])
len(text_or_collection) -> number
range(end) | range(start, end) | range(start, end, step) -> list<number>
type_of(value) -> text
to_text(value) -> text
```

assert_equal preserves the exact/approximate distinction. assert_approx explicitly requests a numerical comparison. Conversion from an exact script value to a binary floating-point evaluation boundary is recorded in the run report and produces warning PSR0701.

5 Complete lexical reference

PhonoScript source is decoded as UTF-8 before tokenization. Source positions use byte offsets internally, while user-facing lines and columns are one-based and columns count Unicode scalar values.

An identifier begins with an underscore or an alphabetic Unicode scalar. It continues with underscores, alphanumeric scalars, or combining marks in the Unicode ranges U+0300–U+036F, U+1AB0–U+1AFF, U+1DC0–U+1DFF, U+20D0–U+20FF, and U+FE20–U+FE2F. Identifier spelling is significant and case-sensitive. The reserved words are:


```
let var fn if else while for in return
true false null and or not command
import export from as
```

command is retained as a parsed legacy bridge, but static analysis rejects unsupported command payloads. New programs should call named builtins and user functions.

Unsigned numeric token spellings are integers, rationals, terminating decimals, and base-ten scientific notation. Signs are unary operators, so $-3/5$ is unary minus applied to exact literal $3/5$. Underscores may separate digits but cannot begin or end a digit run.

```
42          1_000_000          3/7
0.125       .5                 12.5e-3
```

All four forms denote exact rationals. Rational denominators must be nonzero. The absolute written decimal exponent is bounded by 10,000 before exact conversion, and subsequent exact values are subject to the runtime exact-byte limit.

Single- and double-quoted strings have identical semantics. The supported escapes are `\\`, `\"`, `\'`, `\n`, `\r`, `\t`, `\0`, and `\u{H...}` with one through six hexadecimal digits naming a valid Unicode scalar. Single- and double-quoted strings may also contain literal newlines; source locations continue on the following line.

`#` and `//` start line comments. Block comments use `/* ... */` and may nest. A first-line `#!/usr/bin/env phonoscript` shebang is therefore valid trivia.

A newline or semicolon terminates a simple statement. Newlines are soft inside parentheses, brackets, braces used as expression delimiters, argument lists, import lists, list literals, and record literals. A trailing comma is accepted in import, parameter, argument, list, and record lists.

6 Complete grammar

The following EBNF is a compact description of language version 3. Literal tokens such as `"let"` appear in quotes; NEWLINE represents one or more statement-separating line breaks outside a soft-newline context.

```
program      ::= separators? declaration* EOF
declaration  ::= importDecl
               | "export" (letDecl | functionDecl)
               | letDecl | varDecl | functionDecl | statement

importDecl   ::= "import" "{" importItem ("," importItem)* ","? "}"
               "from" STRING terminator
importItem   ::= IDENTIFIER ("as" IDENTIFIER)?
letDecl      ::= "let" IDENTIFIER "=" expression terminator
varDecl      ::= "var" IDENTIFIER "=" expression)? terminator
functionDecl ::= "fn" IDENTIFIER "(" parameters? ")" block
parameters   ::= IDENTIFIER ("," IDENTIFIER)* ","?

statement    ::= ifStmt | whileStmt | forStmt | returnStmt
               | block | commandStmt | expression terminator
ifStmt       ::= "if" condition block
               ("else" (ifStmt | block))?
whileStmt    ::= "while" condition block
```

```

forStmt      ::= "for" IDENTIFIER "in" expression block
returnStmt   ::= "return" expression? terminator
commandStmt  ::= "command" token+ terminator
condition    ::= expression | "(" expression ")"
block        ::= "{" declaration* "}"

expression   ::= assignment
assignment   ::= disjunction ("=" assignment)?
disjunction  ::= conjunction (("or" | "||") conjunction)*
conjunction  ::= equality (("and" | "&&") equality)*
equality     ::= comparison ("==" | "!=") comparison)*
comparison   ::= term ("<" | "<=" | ">" | ">=") term)*
term         ::= factor ("+" | "-") factor)*
factor       ::= unary ("*" | "/" | "%") unary)*
unary        ::= ("+" | "-" | "not" | "!") unary | postfix
postfix      ::= primary (call | index | member)*
call         ::= "(" arguments? ")"
arguments    ::= expression ("," expression)* ","?
index        ::= "[" expression "]"
member       ::= "." IDENTIFIER

primary      ::= NUMBER | STRING | "true" | "false" | "null"
              | IDENTIFIER | "(" expression ")"
              | listLiteral | recordLiteral
listLiteral  ::= "[" arguments? "]"
recordLiteral ::= "{" recordEntry ("," recordEntry)* ","? "}"
recordEntry  ::= (IDENTIFIER | STRING) ":" expression

terminator   ::= ";" | NEWLINE | before-"}" | before-EOF
separators   ::= (";" | NEWLINE)+

```

An assignment target must be a variable name. List elements, record fields, and indexed locations are immutable values and cannot appear on the left side of `=`. Assignment associates right-to-left; every other binary tier in the grammar associates left-to-right. A function or call has at most 255 parameters or arguments.

The scanner reserves the token `->`, but language version 3 assigns it no grammatical or runtime meaning. It is not a return-type annotation or a lambda arrow.

6.1 Operator semantics and precedence

From highest to lowest precedence, postfix calls/indexing/member access are followed by unary operators, multiplicative operators, additive operators, comparison, equality, conjunction, disjunction, and assignment.

Operators	Accepted operands	Result and behavior
+ unary; - unary not, !	number boolean	Identity or negation; exactness is retained. Boolean negation; there is no truthiness conversion.
*, /	numbers	Exact rational arithmetic when both operands are exact; otherwise explicit approximate arithmetic.
%	exact integers	Integer remainder; zero divisor and noninteger operands are errors.
+ binary	two numbers, two texts, or two lists	Arithmetic, text concatenation, or list concatenation. Mixed categories are errors.

Operators	Accepted operands	Result and behavior
< <= > >=	two exact numbers, two approximate numbers, or two texts	Numeric or lexical order. Exact-to-approximate ordering is refused without a common boundary.
== !=	any two values	Structural equality. Exact and approximate numbers are never equal across the boundary.
and &&	booleans	Short-circuit conjunction.
or	booleans	Short-circuit disjunction.
=	mutable variable	Assignment expression; the assigned value is also the expression result.

Indexing is zero-based. A list index is an exact nonnegative integer. A text index selects one Unicode scalar, not one UTF-8 byte or grapheme cluster. A record index is text. Dot access is shorthand for identifier-shaped record keys; use brackets for keys containing spaces or punctuation.

6.2 Bindings, scopes, and functions

`let` creates an initialized immutable binding. `var` creates a mutable binding and may omit its initializer, in which case its initial value is `null`. A same-scope duplicate is an error. A nested scope may shadow an outer binding, but static analysis emits warning PSA1005. Function parameters and for loop bindings are lexical locals.

Functions are closures over the lexical environment at declaration time. Arguments are evaluated left-to-right and passed by value. Lists, records, texts, and numbers are immutable script values. A bare `return` returns `null`; falling off the end of a function does the same. `return` outside a function is rejected statically and defensively at runtime.

Declarations are sequential and are not hoisted. Although the parser admits general postfix call syntax, the current analyzer and runtime require a directly named function or builtin at a call site; functions are not ordinary first-class script values. Language version 3 has no anonymous functions, default or named user arguments, variadic user functions, index/member assignment, `break`, `continue`, exceptions, classes, pattern matching, or user-defined nominal types.

An `if` or loop condition must be Boolean. There are no implicit numeric, text, list, or null truth values. A `for` loop accepts a list or text; text iteration visits Unicode scalars. `while` and `for` consume the common loop-iteration budget.

6.3 Gradual static analysis

The analyzer resolves lexical names, same-scope duplicates, immutability, return placement, known callability and arity, lightweight value categories, record keys, and literal-domain restrictions. It is path-insensitive and gradual: a value not statically known remains *unknown* rather than being executed during checking. Candidate lookup, matrix width, project-state requirements, evaluator admission, and exact engine constraints remain runtime checks.

The static categories are unknown, null, number, Boolean, text, list, record, domain record, and callable. Domain records identify evaluation, learning, ranking, serial, Second-Order, Q-Calculus, mark-data, linear-extension, phonological-form, and generation results so field access can receive more specific checks.

7 Modules and reusable analysis libraries

Only selective local imports are legal:

```
import { build, source as citation } from "./analysis.phont"
```

The imported file must use the `.phont` extension and resolve beneath the canonical `--module-root`. Absolute paths, root escapes including symlink escapes, cycles, missing exports, excessive graph depth, excessive module count, and excessive aggregate source are refused. Imports are immutable bindings; the dependency must explicitly export every selected name.

Only `export let` and `export fn` are valid exports. `export var` is deliberately invalid. A dependency's top level is declaration-only: imports, functions, and immutable initializers that contain no calls or assignments. Put printing, assertions, project mutation, and file effects inside an exported function. Nothing runs merely because the module was imported.

The module graph and entry file share one transaction and one resource budget. If any module fails to parse, check, execute, normalize, validate, or commit a file effect, all project mutations and queued writes are discarded.

Module-root confinement governs source imports only. It is not a filesystem sandbox for `save`, `export_tableau`, or `export_plot`; those explicit paths retain the invoking user's write authority.

8 Diagnostic catalogue

Diagnostic identifiers are stable integration coordinates. The prose may become more specific between compatible releases, so editor extensions and test suites should branch on the code and severity rather than the complete English sentence. Lexer and parser diagnostics use the PSL and PSP families:

Code	Condition
PSL0001	Unexpected source character.
PSL0002	Unterminated string literal.
PSL0003	Invalid string escape or Unicode scalar escape.
PSL0004	Unterminated, possibly nested, block comment.
PSL0005	Invalid or out-of-bound exact numeric literal.
PSP0101	Required token was absent.
PSP0102	Required expression was absent or malformed.
PSP0103	Left side of assignment is not a mutable variable name.
PSP0104	Function declaration exceeds 255 parameters.
PSP0105	Call exceeds 255 arguments.
PSP0106	Parenthesis, bracket, or brace was not closed.
PSP0107	Statement terminator was required.
PSP0108	Legacy command payload was absent.
PSP0109	Unexpected closing delimiter.
PSP0110	Invalid import, export, or module declaration form.

Static analysis uses PSA. All entries below are errors except PSA1005, which is a warning:

Code	Condition
PSA1001	Name cannot be resolved in lexical scope.
PSA1002	Duplicate binding in one scope.
PSA1003	<code>return</code> occurs outside a function.

Code	Condition
PSA1004	Assignment targets an immutable binding.
PSA1005	A nested binding shadows an outer binding.
PSA1006	Duplicate key in one record literal.
PSA1007	Import/export or executable top-level form is invalid for that module position.
PSA1101	Statically known value is not callable.
PSA1102	Statically known function or builtin arity is wrong.
PSA1201	Statically provable type mismatch.
PSA1202	Statically provable domain-admission error.
PSA1203	Statically known domain record lacks a requested field.
PSA1301	Parsed legacy command has no supported execution contract.

Runtime and checked-engine diagnostics use PSR:

Code	Condition
PSR0001	Frontend failure was promoted into the transactional runtime.
PSR0101	Module path, file, root confinement, or decoding failure.
PSR0102	Import cycle.
PSR0103	Module count, depth, or aggregate-source limit exceeded.
PSR0104	Requested imported name is not exported.
PSR0105	Dependency contains an illegal top-level declaration or effect.
PSR0201	Global execution-step limit exceeded.
PSR0202	Loop-iteration limit exceeded.
PSR0203	Function-call depth exceeded.
PSR0204	List, record, range, or generated collection exceeds its bound.
PSR0205	Exact-number storage or numeric resource bound exceeded.
PSR0301	Name is undefined at runtime.
PSR0302	Duplicate runtime name.
PSR0303	Assignment targets an immutable runtime binding.
PSR0304	Runtime value is not callable.
PSR0305	Runtime call arity is wrong.
PSR0401	Runtime value has the wrong type.
PSR0402	Division or remainder by zero.
PSR0403	List, text, or record index is invalid.
PSR0404	Runtime defense found <code>return</code> outside a function.
PSR0405	Unsupported legacy command reached execution.
PSR0450	Script assertion failed.
PSR0501	Phonological object or operation was not formed.
PSR0502	Exact/approximate or another declared domain boundary was crossed illegally.
PSR0503	Unified phonological engine returned a structured refusal.
PSR0601	Staging, validating, committing, or rolling back a file effect failed.
PSR0701	Warning: exact script number was deliberately lowered to an approximate engine boundary.
PSR0901	Internal runtime state violated an invariant.

A script may inspect returned domain-record fields but does not catch diagnostics as values: any error aborts the transaction. Assertions are the intended executable-regression mechanism. Approximation warnings remain in the report even after a successful assertion, preserving the exactness boundary for audit.

9 Projects, tableaux, and explicit violation ledgers

A project can contain multiple named tableaux. Selectors accept a zero-based integer index or, where supported, a displayed name. `tableau_select` also accepts "source" and "target".

```

project_title(text)
project_author(text)
project_description(text)
project_keyword(text)
project_evaluator("OT"|"HG"|"MaxEnt")
project_temperature(positive_number)

dataset_clear()
tableau_new(name, input) -> index
tableau_select(selector)
tableau_copy(source, target)
tableau_name(text)
tableau_input(text)
tableau_notes(text)
tableau_source_locator(text)
tableau_evaluator(text)
tableau_temperature(positive_number)
tableau_ties("retain"|"first"|"unique")

constraints_clear()
constraint_add(name[, weight, definition, stratum]) -> index
constraint_add_unweighted(name[, definition, stratum]) -> index
constraint_remove(selector)
constraint_move(selector, index)
constraint_rank(selector, stratum)
constraint_tie(left, right)
constraint_weight(selector, number)
constraint_definition(selector, text)
constraint_enabled(selector, bool)
constraint_prior(selector, mean, sigma)

candidates_clear()
candidate_add(name, form, violations[, mass, observed]) -> index
candidate_remove(selector)
candidate_move(selector, index)
candidate_name(selector, text)
candidate_form(selector, text)
candidate_mass(selector, positive)
candidate_observed(selector, nonnegative)
candidate_notes(selector, text)
violation_set(candidate, constraint, nonnegative_integer)
violation_get(candidate, constraint) -> number

missing_dependency_add(code, stage, scope, coordinate,
                      message, remedy[, evaluator])

```

Strata and indices are zero-based. A candidate is a candidate for output; the term *output* applies only after an evaluator selects a candidate.

Every violation count is supplied by the phonologist. `candidate_add` requires an explicit vector whose width matches the constraint register. PhonoScript never infers violations from a candidate form, a structured representation, a constraint name, or a descriptive definition. Definitions are prose and are never executed.

Candidate base mass must be positive and observed frequency nonnegative. Enabled HG and MaxEnt constraints require nonnegative weights. Strict OT does not interpret weight magnitude and therefore admits a stored negative weight, but its ranking is controlled by strata. A definition beginning `calc:` is a retired executable-constraint syntax and is refused rather than evaluated.

`constraint_add_unweighted` represents a genuinely unavailable weight; it does not store zero or null. `missing_dependency_add` records why an operation is unavailable. Its stage is formation or admission; its scope is `any_evaluation`, `evaluator`, `learning`, or `exact_certification`. Evaluator scope requires a final OT, HG, or MaxEnt argument. The record survives `.ottab` round trips and causes the checked engine to return the declared structured refusal rather than inventing a parameter.

Schema version 4 reserves unsigned integer 65535 as the unset-cell marker. It is displayed as an em dash and is not a violation count. Admitted counts range from 0 through 65534. Evaluation, learning, comparison, final document validation, and export refuse an unset cell and identify its candidate and constraint coordinates. Before final validation, `violation_get` can expose the internal exact value 65535; scripts must not interpret that sentinel as a phonologist-declared violation.

10 A complete finite script

```
project_title("Coda analysis")
project_evaluator("HG")
tableau_select("source")
tableau_name("Coda")
tableau_input("/bad/")
constraints_clear()
candidates_clear()

constraint_add("*VOICED-CODA", 3/2, "", 0)
constraint_add("IDENT-IO(voice)", 1, "", 1)
candidate_add("faithful", "bad", [1, 0])
candidate_add("devoiced", "bat", [0, 1])

assert_winners(["devoiced"])
save("coda.ottab")
export_tableau("coda.svg", "svg")
```

The two violation vectors are analyst declarations. The interpreter derives the HG costs and selected output. It does not obtain the marks by reading the candidate strings or constraint labels.

11 Evaluation, learning, and serial analysis

```
evaluate() -> record
winners() -> list<text>
winning_forms() -> list<text>
harmony(candidate) -> number (HG or MaxEnt only)
probability(candidate) -> number
assert_winners(names[, message])
assert_winning_forms(forms[, message])
assert_probability(candidate, expected, tolerance[, message])

maxent_learn([iterations]) -> record
infer_ranking() -> record
```

```

harmonic_bounds() -> list<record>
unnecessary_constraints() -> list<text>
mark_data(observed_winner) -> record
constraint_demotion(observed_winner) -> record
partial_ranking_extensions(edges, limit) -> record

serial_side("source"|"target")
serial_start(form)
serial_limit(positive_integer)
serial_clear()
serial_move(from, to, operation, violations)
serial_evaluate() -> record

```

Strict OT groups enabled constraints by declared stratum and compares the resulting vectors lexicographically. Every co-winner is retained. HG multiplies the explicit violation ledger by declared non-negative weights and minimizes total cost. Finite MaxEnt assigns candidate y with base mass $\rho(y)$, cost $C(y)$, and temperature T the unnormalized mass $\rho(y) \exp(-C(y)/T)$ and calculates an input-specific normalizer using a max-shifted log-mass computation.

Strict OT has no scalar Harmony coordinate. Consequently, `evaluate().rows[*].harmony` is `null` under OT, and `harmony(candidate)` returns the structured formation diagnostic PSR0501; the ordered violation profile and winner relation remain available. For HG and MaxEnt, a candidate's weighted cost is an exact rational when every enabled weight is exact. A cost containing an explicitly approximate weight is approximate. MaxEnt exponentials, normalized probabilities, and numerical learning remain approximate engine results.

The MaxEnt learner minimizes the finite Gaussian-prior negative log likelihood by projected descent with monotone backtracking. Numerical learning and displayed probabilities are approximate engine results. Exact finite MaxEnt law comparison is a separate symbolic operation.

Serial analysis applies the same selected OT or HG grammar to every declared GEN1 candidate set. Identity convergence, a missing local candidate set, co-winner refusal, cycle, and step-limit exhaustion are distinct results. MaxEnt serial sampling is refused until a sampling law is declared.

12 Second-Order comparison and Q-Calculus

```

second_query(text)
second_answer_sort(text)
second_scope(text)
second_transformation(text)
second_transport(text)
second_layout(text)
second_mode("exact"|"approximate"|"grid")
second_tolerance(nonnegative)
second_grid_step(positive)
second_response_domain("terminal"|"trajectory")
second_normalizer(text)
second_layers(source, target)
second_layer_transport(text)
second_consumer(mode, consumer)
second_compare() -> record

q_ranking_space() -> record
q_clone(constraint) -> record
typology() -> record

```


A Second-Order comparison registers the query, answer sort, support, transformation, transport, response domain, exactness mode, normalizer policy, scientific layers, and optional later consumer. The engine calculates source and target responses independently and only then applies the declared transport to the source response. Each calculation uses that tableau's `tableau_evaluator` and `tableau_temperature` override when present, falling back independently to the project evaluator and temperature. The two sides may therefore use different admitted evaluator contexts. A transport aligns responses; it does not generate the target answer.

The result is `PRESERVED` with a certificate, `DISCREPANCY` with every differing response coordinate, or `NOT EVALUATED` with a stable code, contract stage, coordinate, explanation, and remedy. Missing support, transport, normalizer policy, layer bridge, or serial ledger is never translated into false, zero, NaN, or an empty response.

Q-Calculus ranking-space operations count exact compatible rankings with arbitrary-precision integers and reduce their shares as exact rational numbers. The registered operation is presently limited to strict OT, the `retain_all` co-winners policy, no project-level a-priori ranking relations, exactly aligned nonempty tableaux, 1 through 60 enabled constraints, and at most 63 candidates per tableau. Disabled constraints are excluded from the ranking carrier. The memoized search has a default budget of 2,000,000 charged states; exhaustion returns `PE-Q-STATE-BUDGET` instead of an approximate answer. A ranking share is not relabelled as a token probability without an explicit measure and response law.

13 Structured phonology and bounded GEN

The text generator helpers return finite lists of forms:

```
generator_identity
generator_delete
generator_insert
generator_substitute
generator_swap
```

`candidates_from_forms` imports those forms only when the analyst supplies one explicit vector for every form or one explicit shared vector.

The typed record model preserves segment features, morphology, syllables, moras, feet, prosodic words, tone tiers and associations, forms, correspondence graphs, and derivation steps. Native finite-GEN operations are identity, deletion, insertion, feature change, metathesis, affixation, reduplication, syllabification, stress assignment, and tone assignment.

`finite_gen` returns a record containing status, completeness flag, support claim, reasons, statistics, candidates, and an execution-local handle. Status is `complete` only relative to the declared finite domain and support claim. Exploratory support or an exhausted resource bound is `truncated`; malformed or inadmissible input is `refused`.

`generation_to_tableau` requires the generation result and an explicit violation ledger at the import boundary. It refuses incomplete support by default. Its optional import flag admits the bounded retained support and labels it accordingly.

`candidate_add_structured` validates and preserves structured correspondence and derivational identity. Its required arguments include the candidate name, structured record, and analyst-supplied violation vector; mass and observed count are optional.

13.1 Structured-form literal reference

`phonological_form(label, segments[, structure])` assigns stable zero-based identifiers in list order. Each segment is either text or a record `{symbol, features}`. A feature value may be Boolean (positive or negative), null (unspecified), an exact 32-bit integer, arbitrary symbol text, or an explicit record whose `kind` is `positive`, `negative`, `unspecified`, `symbol`, or `integer`. The optional structure admits only these coordinates:

Coordinate	Record fields
<code>morphemes[]</code>	<code>label</code> , <code>kind</code> , and zero-based <code>segments</code> ; standard kinds are <code>root</code> , <code>stem</code> , <code>prefix</code> , <code>suffix</code> , <code>infix</code> , <code>reduplicant</code> , and <code>clitic</code> , while another text is retained as user-named.
<code>prosody.syllables[]</code>	Segment-index lists <code>onset</code> , <code>nucleus</code> , and <code>coda</code> ; optional stress is <code>unstressed/none</code> , <code>secondary</code> , or <code>primary</code> .
<code>prosody.moras[]</code>	Required syllable index and optional segment-index bearers.
<code>prosody.feet[]</code>	Syllable-index list and optional syllable-index head, which may be null.
<code>prosody.words[]</code>	Syllable-index list and morpheme-index list.
<code>tiers[]</code>	Tier name, nodes, and associations. A node may be a tone level, contour, or symbol, or an explicit tone/feature/symbol record.
<code>tiers[].associations[]</code>	Tier-node index plus target record <code>{kind, id}</code> ; <code>kind</code> is <code>segment</code> , <code>syllable</code> , or <code>mora</code> .

All structural references are validated for existence, uniqueness, and well-formed incidence. Unknown fields are errors rather than ignored metadata. A tone level must fit a signed byte; a contour is a list of such levels. Metadata that is not one of these linguistic coordinates belongs in the complete structured-candidate record or the surrounding analysis, not in an undeclared form field.

13.2 Finite-GEN specification reference

The second argument of `finite_gen` has required fields `name`, `operations`, `domain`, `resources`, and `support_claim`; `deduplication` defaults to `structured-representation`. Each operation declaration has `id`, an operation record, and this semantic bound:

`max_applications_per_candidate`

The coordinate bounds the declared finite grammar; it is not a wall-clock timeout.

Operation kind	Required operation fields
<code>identity</code>	No additional fields.
<code>delete</code>	Segment selector.
<code>insert</code>	Segment-template inventory and insertion sites.
<code>feature-change</code>	Segment selector, feature name, and nonempty feature-value list values.
<code>metathesis</code>	Segment selector and exact nonnegative <code>max_distance</code> .
<code>affix</code>	Morpheme template <code>{label, kind, segments}</code> and insertion site.
<code>reduplicate</code>	Reduplication domain and site prefix or suffix.
<code>syllabify</code>	specification with nucleus selector, maximum onset/coda sizes, and empty-onset/coda Booleans.
<code>assign-stress</code>	specification with primary position and secondary-stress policy.
<code>assign-tone</code>	specification with tier name, tone inventory, targets, and association pattern.

A segment selector has `kind all`, `at` with positions, `symbol`, `feature` with name and value, `morpheme-kind`, and `or` with a selector list, or `not` with one selector. Insertion sites are every boundary, initial, final, exact boundaries, before a selector, or after a selector. Affix sites are prefix, suffix, exact boundaries, or after the first selected segment. Reduplication domains are the whole form, a half-open segment range, the first syllable, or a named morpheme.

Primary stress positions are any, initial, final, penultimate, or an exact syllable index. Secondary policies are none, alternating left-to-right, or alternating right-to-left. Tone targets are segments, syllables, or moras; patterns are one-per-target, spread-single, or floating.

The domain and resource coordinates are:

```
domain:    max_derivation_steps, max_segments_per_form
resources: max_candidates, max_operation_expansions,
           max_variants_per_application
```

A support claim is `exploratory`, or one of the following with its required statement and, for the second form, a certificate identifier:

```
complete-for-declared-domain
complete-by-certificate
```

The certificate form records an external proof obligation; a nonempty identifier does not make the engine an independent checker of that external proof.

Deduplication has three values:

```
surface-form    structured-representation    preserve-derivations
```

Surface deduplication is sound only when no constraint or query consumes correspondence, morphology, prosody, or derivation. Preserve-derivations is required when construction history belongs to the response type.

13.3 Complete structured-candidate records

`candidate_add_structured` also accepts a fully authored candidate record. Its top-level fields are exact integer `id`, presentation `label`, exact `underlying_form` and `surface_form` identifiers, a `forms` record keyed by those identifiers, `correspondences`, `derivation`, and text-to-text metadata. Each stored form contains `id`, `label`, a tagged `role`, segments with explicit identifiers and origins, morphemes, prosody, and tiers.

A correspondence graph contains `id`, `label`, tagged `kind`, source/target form identifiers, and `links`. Standard kinds are input-output, base-reduplicant, output-output, and sympathy; a user-named relation carries its name. Each link has an identifier and lists of tagged source and target nodes. Node kinds are segment, morpheme, syllable, and mora. An empty source denotes insertion relative to that graph; an empty target denotes deletion. The validator checks that every referenced form and node exists and that the graph is well formed.

Derivation entries record `operation_id`, tagged `operation_class`, and structural edits. The admitted edit kinds are identity, delete, insert, feature-change, metathesis, affix, reduplicate, syllabify, assign-stress, and assign-tone, with their affected identifiers and coordinates. This record is appropriate when correspondence or derivational identity is analyst-authored. When finite GEN produced the candidate, prefer its execution-local handle and `generation_to_tableau` so the native record is transferred without a hand transcription. In both routes, the violation vector remains an independent phonologist-supplied argument.

14 Complete builtin reference

The 103 signatures below are the complete registered builtin surface for language version 3. A selector is either a zero-based exact integer index or a name accepted by the relevant project object. Optional arguments appear in square brackets. Mutation functions return `null` unless another return type is shown.

14.1 General values and assertions

Signature	Result	Contract
<code>print(values...)</code>	<code>null</code>	Renders values to captured standard output; bounded by output bytes.
<code>assert(condition[, message])</code>	<code>Boolean</code>	Requires a <code>Boolean</code> and fails with PSR0450 when false.
<code>assert_equal(left, right[, message])</code>	<code>Boolean</code>	Structural equality; preserves exact/approximate distinction.
<code>assert_approx(left, right, tolerance[, message])</code>	<code>Boolean</code>	Explicit finite numerical boundary; tolerance must be nonnegative.
<code>len(value)</code>	<code>exact number</code>	Unicode-scalar count for text; item or field count for collections.
<code>range(end)</code>	<code>list</code>	Exact integer range $[0, end)$.
<code>range(start, end[, step])</code>	<code>list</code>	Exclusive end; nonzero exact integer step.
<code>type_of(value)</code>	<code>text</code>	Runtime type name such as <code>exact-number</code> or <code>record</code> .
<code>to_text(value)</code>	<code>text</code>	Stable human rendering, not a parser round trip.
<code>project_restore_v2(json)</code>	<code>null</code>	Internal lossless restore used by <code>--emit</code> ; validates the complete current document.

`assert_approx` and `assert_probability` are the two named builtins that may lower an exact script number into the current binary floating-point assertion boundary. Each lowering records coordinate, exact value, engine value, and source span and emits PSR0701.

14.2 Project and tableau mutation

Signature	Effect or result
<code>project_title(text)</code>	Set project title.
<code>project_author(text)</code>	Set project author.
<code>project_description(text)</code>	Set project description.
<code>project_keyword(text)</code>	Add one project keyword.
<code>project_evaluator(text)</code>	Set default evaluator: OT, HG, or MaxEnt.
<code>project_temperature(positive)</code>	Set default MaxEnt temperature.
<code>dataset_clear()</code>	Remove all additional dataset tableaux; source and target remain.
<code>tableau_new(name, input)</code>	Append a dataset tableau and return its zero-based dataset index.
<code>tableau_select(selector)</code>	Select source, target, dataset index, or supported name.
<code>tableau_copy(source, target)</code>	Copy tableau content into an existing selected slot.

Signature	Effect or result
<code>tableau_name(text)</code>	Set current tableau display name.
<code>tableau_input(text)</code>	Set current input representation.
<code>tableau_notes(text)</code>	Set notes.
<code>tableau_source_locator(text)</code>	Set stable provenance locator.
<code>tableau_evaluator(text)</code>	Set current-tableau evaluator override.
<code>tableau_temperature(positive)</code>	Set current-tableau temperature override.
<code>tableau_ties(text)</code>	Set <code>retain</code> , <code>first</code> , or <code>unique</code> .

`retain` keeps every co-winner, `first` selects the first listed co-winner, and `unique` refuses an unresolved tie. The policy is recorded in the project; it is not a display preference.

Moving a candidate changes the order consulted by `first`. Moving a constraint is currently a reranking operation: after reordering the column and its aligned cells, it replaces prior strata, including ties, with successive column-order strata. Use `constraint_rank` and `constraint_tie` after a move when the intended ranking contains ties. Tableau display names need not be unique; name lookup selects the first match, and `tableau_copy` copies the display name. Stable indices avoid that ambiguity. `tableau_evaluator("inherit")` clears an evaluator override, but no corresponding temperature-inheritance operation exists in language version 3.

14.3 Constraints, candidates, and violation cells

Signature	Effect or result
<code>constraints_clear()</code>	Remove all constraints and corresponding matrix columns.
<code>constraint_add(name[,weight,definition,stratum])</code>	Add a weighted constraint; return index.
<code>constraint_add_unweighted(name[,definition,stratum])</code>	Add a genuinely weightless/unavailable constraint; return index.
<code>constraint_remove(selector)</code>	Remove constraint and its entire violation column.
<code>constraint_move(selector,index)</code>	Move the column and preserve its cells; then reset all strata to column order.
<code>constraint_rank(selector,stratum)</code>	Assign zero-based OT stratum.
<code>constraint_tie(left,right)</code>	Put the named constraints in one stratum.
<code>constraint_weight(selector,number)</code>	Set explicit weight.
<code>constraint_definition(selector,text)</code>	Set nonexecuting prose definition.
<code>constraint_enabled(selector,bool)</code>	Enable or disable constraint.
<code>constraint_prior(selector,mean,sigma)</code>	Set Gaussian prior with positive σ .
<code>candidates_clear()</code>	Remove all candidates and violation rows.
<code>candidate_add(name,form,marks[,mass,observed])</code>	Add candidate with explicit marks; return index.
<code>candidate_add_structured(name,record,marks[,mass,observed])</code>	Add structured candidate with explicit marks; return index.
<code>candidate_remove(selector)</code>	Remove candidate and its complete row.
<code>candidate_move(selector,index)</code>	Move candidate row without changing its analysis.
<code>candidate_name(selector,text)</code>	Rename candidate identity label.
<code>candidate_form(selector,text)</code>	Set displayed candidate form.
<code>candidate_mass(selector,positive)</code>	Set positive base mass.
<code>candidate_observed(selector,nonnegative)</code>	Set observed count.

Signature	Effect or result
<code>candidate_notes(selector, text)</code>	Set candidate notes.
<code>violation_set(candidate, constraint, count)</code>	Set analyst-declared count from 0 through 65534.
<code>violation_get(candidate, constraint)</code>	Return exact count; refuse an unset cell.

`missing_dependency_add(code, stage, scope, coordinate, message, remedy[, evaluator])`

The stage is formation or admission. Scope is any_evaluation, evaluator, learning, or exact_certification; evaluator scope requires OT, HG, or MaxEnt. This function records an intentional absence. It is preferable to inserting a fabricated zero weight, guessed violation count, or empty candidate set.

14.4 Evaluation and learning

Signature	Result
<code>evaluate()</code>	Evaluation record described below.
<code>winners()</code>	Candidate names selected by the current evaluator and tie policy.
<code>winning_forms()</code>	Forms of the selected candidates.
<code>harmony(candidate)</code>	Exact HG/MaxEnt weighted cost where possible; undefined for strict OT.
<code>probability(candidate)</code>	Approximate finite MaxEnt probability; null/formation refusal outside MaxEnt.
<code>assert_winners(names[, message])</code>	Check the complete selected candidate-name list.
<code>assert_winning_forms(forms[, message])</code>	Check the complete selected form list.
<code>assert_probability(candidate, expected, tolerance[, message])</code>	Explicit approximate probability check.
<code>maxent_learn([iterations])</code>	Numerical learning record; iterations positive when supplied.
<code>infer_ranking()</code>	Exact finite ranking-inference record from stored observations.
<code>harmonic_bounds()</code>	List of pairwise harmonic-bounding records.
<code>unnecessary_constraints()</code>	Constraint names unnecessary for the current finite winner relation.
<code>mark_data(observed_winner)</code>	Comparative mark-data record.
<code>constraint_demotion(observed_winner)</code>	One constraint-demotion result.
<code>partial_ranking_extensions(edges, limit)</code>	Bounded linear extensions and truncation status.

`evaluate()` returns fields evaluator, winners, winning_forms, tie_unresolved, and rows. Each row has these coordinates:

candidate form harmony probability winner fatal_constraint

Under strict OT, harmony is null because the evaluator compares a lexicographic profile rather than a scalar cost. Under non-MaxEnt evaluators, probability is null.

`maxent_learn()` returns approximate weights, exact iterations, Boolean converged, approximate negative_log_likelihood, and approximate maximum_gradient. It also

writes learned approximate weights to every tableau with the same constraint-name register. The default maximum is 10,000 iterations; an explicit bound must be from 1 through 10,000,000. `infer_ranking()` returns the constraint-name order and exact explored states, and writes those strata to matching registers. Unlike Q ranking-space enumeration, ordinary ranking inference may consume declared a-priori dominance relations.

Each `harmonic_bounds()` item contains `input`, `observed_candidate`, and `bounding_candidate`; `unnecessary_constraints()` returns names. `mark_data()` returns the constraint list, comparative rows with winner/loser candidates and their mark sets, and discarded rows with reasons. `constraint_demotion()` returns status learned with strata, trace-step count, and unresolved pairs, or status inconsistent with code, message, trace-step count, and conflicting-data indices. A learned result writes its strata to the current tableau.

`partial_ranking_extensions()` returns status complete and orders, status truncated with orders, limit, and message, or status refused with code and message. Input edges are two-item lists `[higher, lower]`; the positive limit cannot exceed the collection ceiling.

14.5 Serial, Second-Order, and Q operations

Signature	Effect or result
<code>serial_side("source" "target")</code>	Select independent serial ledger.
<code>serial_start(form)</code>	Set initial form for that ledger.
<code>serial_limit(positive_integer)</code>	Set derivation step bound.
<code>serial_clear()</code>	Clear moves on the selected side.
<code>serial_move(from,to,operation,marks)</code>	Add one declared GEN1 move with explicit marks.
<code>serial_evaluate()</code>	Status, path, selected moves, and stopping/refusal data.
<code>second_query(text)</code>	Set winner set, surface winner set, complete order, probability law, or candidate support.
<code>second_answer_sort(text)</code>	Set registered answer-sort label.
<code>second_scope(text)</code>	Set support/case scope.
<code>second_transformation(text)</code>	Describe declared upper transformation.
<code>second_transport(text)</code>	Set lower answer transport.
<code>second_layout(text)</code>	Set overlay, delta sidecar, or expanded paired layout.
<code>second_mode(text)</code>	Set exact, approximate, or grid comparison.
<code>second_tolerance(nonnegative)</code>	Set approximate tolerance.
<code>second_grid_step(positive)</code>	Set grid step.
<code>second_response_domain(text)</code>	Set terminal or complete trajectory.
<code>second_normalizer(text)</code>	Set independent or explicitly shared policy.
<code>second_layers(source,target)</code>	Set scientific layers.
<code>second_layer_transport(text)</code>	Set explicit layer bridge label.
<code>second_consumer(mode,name)</code>	Set direct or later consumer.
<code>second_compare()</code>	Typed comparison record with status, answers, discrepancies, refusal, and certificate.
<code>q_ranking_space()</code>	Exact total ranking count, answer classes, states, and measured elapsed time.
<code>typology()</code>	Alias of <code>q_ranking_space</code> .
<code>q_clone(constraint)</code>	Support and uniform-ranking-share audit after exact constraint cloning.

`second_compare()` returns status, a convenience Boolean conservative, independently calculated `source_answer` and `target_answer`, `transported_source_answer`, the complete discrepancies list, refusal, certificate, and source/target normalizer fields. Always branch on status; a false convenience Boolean can mean discrepancy or non-evaluation.

`serial_evaluate()` returns `path`, `selected_operations`, `textual_stopping_coordinate` `stopped`, and `Boolean` `formed`. A configured limit must be from 1 through 1,000,000. The record distinguishes a formed stopped path from a failed local competition; it does not replace that distinction with an empty path.

`q_ranking_space()` represents arbitrarily large counts as exact script integers and returns `total_rankings`, `outcomes`, `dynamic_states`, `completion_states`, `state_budget`, and `approximate_elapsed_seconds`. Each outcome has the complete tableau answer and exact ranking count.

`q_clone()` returns support and share conservativity Booleans, exact before/after totals, the state budget, and exact rational shifts represented by four numerator/denominator fields. It does not reinterpret uniform ranking share as observed token probability.

14.6 Text generation and structured phonology

Signature	Result or effect
<code>segments(text)</code>	List of Unicode scalar strings.
<code>unique(list)</code>	First-occurrence-preserving duplicate removal.
<code>generator_identity(form)</code>	Singleton identity form.
<code>generator_delete(form)</code>	Finite single-deletion forms.
<code>generator_insert(form, items)</code>	Finite single-insertion forms.
<code>generator_substitute(form, items)</code>	Finite single-substitution forms.
<code>generator_swap(form)</code>	Finite adjacent-swap forms.
<code>candidates_from_forms(forms, marks)</code>	Import forms only with one explicit vector per form or one shared vector.
<code>phonological_form(id, segments[, structure])</code>	Validated structured-form domain record.
<code>finite_gen(form, spec)</code>	Complete, truncated, or refused bounded generation record.
<code>generation_to_tableau(result, marks[, allow_truncated])</code>	Import generated candidates only at an explicit violation-ledger boundary.

`finite_gen` consumes structured records describing a finite domain and bounds. Supported native operations are identity, deletion, insertion, feature change, metathesis, affixation, reduplication, syllabification, stress assignment, and tone assignment. Its result carries `status`, `complete`, `support claim`, `reasons`, `statistics`, `candidates`, and an execution-local handle. Only the handle from the same successful run may be used for native structured import.

14.7 Files and publication export

Signature	Result
<code>save(path)</code>	Queue current normalized project as <code>.ot.tab</code> ; return destination text.
<code>export_tableau(path, format[, second_order])</code>	Queue cropped tableau SVG, PNG, or PDF; return destination text.
<code>export_plot(path, format)</code>	Queue cropped plot SVG, PNG, or PDF; return destination text.

The extension and format argument must agree. The runtime stages all queued effects, backs up existing destinations, commits only after the project passes validation, and rolls back committed destina-

tions if a later staged rename fails. Scripts are code: review destinations in an untrusted script before execution.

15 Scripting patterns for reproducible analyses

15.1 Build, calculate, and assert

A research script should separate analyst declarations from calculated results. First declare provenance, evaluator, constraints, candidates, and the complete violation ledger. Then call the engine and assert the bounded result. Save or export only after the assertions:

```
tableau_source_locator("Author year; page/table; transcription ceiling")
tableau_evaluator("OT")
tableau_ties("retain")

// Constraints, candidates, and every violation are analyst declarations.
let result = evaluate()
assert_equal(result.evaluator, "OT")
assert_equal(result.winners, ["devoiced"])

save("analysis.ottab")
export_tableau("analysis.svg", "svg")
```

If an assertion or export fails, the project and every queued destination roll back. This makes one script both an analysis recipe and an executable regression. Do not insert expected winners, probabilities, or costs into the project as though they were calculated values; expectations belong in assertions.

15.2 Status-first handling of typed results

Records from finite GEN, serial evaluation, Second-Order comparison, ranking inference, and Q operations retain status or formation information. Inspect that coordinate before reading branch-specific fields:

```
let comparison = second_compare()

if comparison.status == "PRESERVED" {
  assert(comparison.conservative)
} else if comparison.status == "DISCREPANCY" {
  assert(len(comparison.discrepancies) > 0)
} else {
  assert_equal(comparison.status, "NOT EVALUATED")
  print(comparison.refusal.code, comparison.refusal.remedy)
}
```

The convenience Boolean `conservative` is not an outcome sum type: it is false for a discrepancy and may also be false when the relation never formed. The status distinction is therefore mandatory. Likewise, `finite_gen().complete` is meaningful with its textual status and support claim; a truncated carrier is not silently a complete GEN.

15.3 Exact and approximate assertions

Use `assert_equal` for exact integers, rationals, strings, lists, records, winner sets, and exact ranking counts. Use `assert_probability` or `assert_approx` only when the intended claim is numerical approximation:

```
assert_equal(q_ranking_space().total_rankings, 6)
assert_probability("deletion", 0.73103432, 0.00000001)
```

The second line creates an explicit exact-to-engine boundary for its expected number and records PSR0701. A tolerant assertion is not an exact MaxEnt-law certificate; exact finite law preservation belongs to the typed Second-Order symbolic comparison path.

15.4 Reusable modules

Keep source locators and pure declaration data in exported immutable values, and put mutations in an exported function:

```
// final_devoicing.phont
export let citation = "Kager 1999, pp. 16--17"
export fn build() {
  tableau_name("Final devoicing")
  // declare constraints, candidates, and analyst-supplied marks
}

// study.phont
import { citation, build } from "./final_devoicing.phont"
print(citation)
build()
assert_winners(["devoiced"])
```

Run a module graph with an explicit root:

```
phonoscript --module-root analyses analyses/study.phont
```

Imports are not package downloads and do not execute dependency top levels. This keeps provenance and reusable constructors local, inspectable, and inside the same all-or-nothing transaction.

15.5 Multi-tableau projects

Source and target are persistent distinguished tableaux; additional datasets are appended with `tableau_new`. Build or import each finite competition, select it explicitly, and evaluate it under its own override:

```
let first = tableau_new("Input one", "/x/")
tableau_select(first)
tableau_evaluator("HG")
// declare its constraints, candidates, and marks

let second = tableau_new("Input two", "/y/")
tableau_select(second)
tableau_evaluator("MaxEnt")
// declare its independent finite competition
```

Names are editable display labels; stable object identities are stored in the `.ottab` document. Prefer returned indices or unambiguous names in scripts, and always select a tableau before mutating its cells.

16 The `.ottab` document format

An `.ottab` file is a self-contained UTF-8 JSON project with format identity `convolgen-analysis` and `format_version` 4. Stable project, tableau, constraint, and candidate identities are independent of editable display labels.

The loader is current-version-only. It does not migrate an older schema or guess the meaning of unknown fields; convert such a file with the version that created it or reconstruct the declared analysis explicitly.

The document stores project metadata; default and per-tableau evaluator settings; source, target, and dataset tableaux; independent serial ledgers; typed Second-Order and Q contracts; plots and presentation settings; source locators; regression expectations; and structured missing dependencies. A regression expectation is checked only after the engine calculates the result. For the dissertation corpus, `source_locator` contains a stable English/Portuguese-shared LaTeX label plus an explicit fidelity tag; editable display names are not used as the provenance key.

Exact scalars use canonical numerator and denominator strings. Approximate scalars store a decimal center and a boundary record with method, provenance, working precision, and a certified absolute error when one exists. Absence of a certified error means uncertified, not zero error. A cached or displayed floating-point probability never becomes exact merely because it is persisted.

Exact finite MaxEnt comparison represents unnormalized masses as formal exponential polynomials with exact rational coefficients and compares normalized laws by cross multiplication. Candidate renaming, mass-preserving fusion, and distinct normalizers are explicit contract cases. Tolerance and grid judgments remain separate modes.

On load and save, the document validator checks version, identities, support, scalar domains, positive temperatures and base masses, nonnegative observations and tolerances, positive grid and serial limits, unique dataset identities, rectangular violation matrices, unset cells, and serial/Second-Order bounds. Saves write and synchronize a sibling temporary file before replacement, so a failed ordinary single-writer save does not expose a partial project. The temporary sibling has a fixed name, so concurrent writers to the same destination are outside the save contract.

17 File effects and export

```
save(path) -> text
export_tableau(path, "svg"|"png"|"pdf"[, second_order]) -> text
export_plot(path, "svg"|"png"|"pdf") -> text
```

File effects are queued transactionally. SVG is the editable vector master; PDF retains vector text and line work; PNG is rasterized from the same scene. Exports are cropped to content. The native renderer implements the material visual grammar of the bundled `secondordertableau.sty` package and never emits an intermediate `.tex` file.

18 Implemented capabilities and boundaries

Capability	Status	Checked boundary
Strict OT with tied strata and all co-winners	Implemented	Declared finite supports; complete order remains distinct from winner identity
HG weighted costs and gang effects	Implemented	Exact ledger and stored weights; no inferred violations
Finite MaxEnt with positive base mass	Implemented	Stable input-specific normalization; numerical probability remains approximate
MaxEnt learning	Implemented	Finite observed counts and Gaussian priors; no empirical-warrant claim
Ranking inference	Implemented	Exact finite inference may use declared a-priori dominance
Serial OT/HG	Implemented	Identity, ties, missing local support, cycles, and limits are distinct outcomes
Second-Order comparison	Implemented	Typed registered responses, transports, layers, normalizers, consumers, and refusal
Q-Calculus typology and cloning	Implemented	Strict-OT total-ranking counts with no a-priori relation; shares are not token probabilities by default
Structured phonology and finite GEN	Implemented	Complete, truncated, and refused status relative to declared bounds
Continuous HG over a continuous candidate domain	Not implemented	Finite grid comparison is sampled agreement, not a continuous-domain certificate
Local selective modules	Implemented	Explicit exports, immutable imports, declaration-only dependencies, canonical root confinement, and bounded acyclic graphs; no remote loader or package registry
OTSoft-style tabular exchange	Interoperable	Shared finite tableau data, not spreadsheet macro state or formatting side effects
SVG, PDF, and PNG	Implemented	One cropped native vector scene; platform-specific appearance is bounded by bundled fonts

19 Validation and source-replication scope

The current corpus gate requires exactly 24 distinct authored `.phon.t` programs, runs each from a blank project, and checks that it is self-asserting, shebang-bearing, executable, and free of an embedded restore payload. Adding a legitimate validation program therefore also requires an intentional update to that fixed gate. The reference-project gate checks nine current `.ot.tab` fixtures, decoding, validating, and evaluating their declared oracle boundaries. These are bounded named corpora rather than an open-ended claim that every file in the main research repository has been replicated.

Seven labels are kept distinct:

- **Source-exact finite replication** uses the printed finite support, parameters, and answer without adding analyst data.
- **Source-bounded normalized correction** preserves the printed finite ledger and identifies a source arithmetic or normalization problem before calculating the corrected result on that declared support.
- **Source-exact ledger transcription with structured refusal** uses printed finite records but declines an evaluator answer when a required parameter is absent.

- **Bounded reconstruction** names the finite-support, row, or representation restriction required for replay.
- **Synthetic or derivative check** exercises an implemented mechanism but is not a transcription of the named source analysis.
- **Source-inspired check** tests a named mechanism without the provenance needed to claim source replication.
- **Refusal case** passes when the engine identifies a missing dependency instead of fabricating a value.

19.1 Source-exact finite cases and exact ledger transcription

Source and asset	Checked result	Ceiling
Kager 1999, printed pp. 16–17; kager-coda-voicing.phont Pater 2008, Tableau (13); pater-harmonic-grammar.phont and pater-hg.ottab	Dutch /bɛd/ selects [bɛt]; the printed English reranking selects [bɛd] [da] wins the first panel; [tat] wins the second with the printed costs	The two displayed finite competitions, not Kager’s full textbook grammar Both printed panels only
Anttila and Cho 1998, Table (11); anttila-cho-linking-r.phont	Three total rankings under *CODA >> ONSET; all four printed competition counts [3, 0], [1, 2], [2, 1], and [3, 0]	Exact total-ranking enumeration and printed competition counts; reduced shares require the declared uniform ranking interpretation and are not token rates by default
Tessier 2017, physical p. 16, Tableaux (14)–(15); tessier-hg-maxent.phont and tessier-hg-maxent.ottab	The printed support, weights, violations, and costs give exact costs 11, 1, and 2; HG selects deletion, and finite MaxEnt normalization gives approximately 0.00003319, 0.73103432, and 0.26893250	Corrects the printed decimal for $\exp(-11)$ and supplies the omitted normalizer; it does not claim that the source printed a normalized law, a complete GEN, or learned weights
Goldwater and Johnson, Tables 2–3; goldwater-johnson-table-2.phont and goldwater-johnson-finnish-ledger.ottab	Exact printed-ledger transcription: four word classes, eleven constraint columns, and the printed difference rows	This is not evaluator replication: the learned weight vector is absent, so Table 4 probability replay receives a structured refusal

19.2 Bounded reconstructions

Asset	Checked result	Ceiling
mccarthy-rimi.phont	McCarthy 2000 Tableau (17), printed rows a–b, and the stated spreading-first GEN1 obstruction	Excludes shaded or unprinted cells; not a complete Rimi grammar
dissertation-complete.phont dissertation-complete.ottab	Thirty-nine bounded records anchored to stable English/Portuguese-shared LaTeX labels; 32 winner or modal oracles and seven intentionally neutral records	The tagged stored records, not every dissertation theorem or cited work; names such as H.1 are navigation labels rather than provenance keys

Asset	Checked result	Ceiling
dissertation-second-order.phont	Independently calculated source and target responses and a two-coordinate complete-order discrepancy under the stored identity transport	The stored contract and answer type only
dissertation-second-order.ottab	Exact finite symbolic law comparison under a declared mass-preserving fusion	No continuous-support or undeclared-normalizer claim
maxent-fusion-second-order.phont		

The dissertation fidelity tag in each `source_locator` states the record’s evidential ceiling:

- `fidelity:exact`: a direct replay of the labeled finite record;
- `fidelity:exact-transform`: the recorded answer-preserving reparameterization, scaling, or specialization;
- `fidelity:neutral-ledger`: support and cells without an evaluator oracle;
- `fidelity:snapshot-only`: one bounded state rather than a complete analysis;
- `fidelity:flattened-panel`: a displayed lane or path converted to an ordinary tableau, not a serial-derivation replay; and
- `fidelity:source-panel-only`: only the named source panel, not the complete Second-Order judgment or refusal.

The partial `prince-smolensky-ot.ottab` Berber fixture is source-inspired rather than a source replication: it checks the finite ONS/HNUC winner and fatal-mark mechanism, but does not carry the independent transcription and stable locator required for a stronger claim.

The Kager basic-syllable, harmonic-bounding, and tie/order programs are small derivative conformance cases, not chapter-complete transcriptions. The Anttila-shaped Q and Finnish ranking-space assets are synthetic. The generic finite-MaxEnt and serial-syllabification assets test engine mechanisms rather than reproduce Goldwater–Johnson probabilities or McCarthy’s complete Rimi grammar.

Refusal coverage includes an unpublished fitted weight vector, incomplete generated support, missing transport or scientific-layer bridge, absent serial ledger, absent normalizer policy, unset violation cell, and MaxEnt serial sampling without a declared sampling law.

The automated gates include focused unit, command-line, cross-interface, property, stress, fixed-corpus, and reference-project tests. Where an independent native transcription exists, cross-interface checks compare it with both the authored PhonoScript program and the `.ottab` project. The dissertation fixture is also checked for byte identity with the copy distributed with ConvalGEN.

20 Performance and scientific limit

Ordinary evaluators are linear in the finite violation matrix apart from candidate sorting. The ranking-space implementation memoizes constraint-choice and survivor states. The fixed release benchmark covers all $16! = 20,922,789,888,000$ rankings for four six-candidate tableaux and separate batches of 50,000 finite-MaxEnt evaluations and 20,000 complete typed Second-Order comparisons. The encoded budgets are release gates on the machine that runs them, not universal hardware guarantees.

A passing suite supports the checked finite calculations, exactness boundaries, format behavior, and structured refusals. It does not choose the scientifically appropriate query, make a declared support complete, infer violation counts, convert finite records into population evidence, settle historical priority, or establish exhaustive reproduction of the phonological literature. It also does not implement or certify the dissertation’s general continuous-HG optimization results: there is no continuous candidate domain, general KKT or persistence solver, support-threshold engine, or contact/inversion solver.

21 Security, contribution discipline, and license

Treat a `.phon.t` or `.ot.tab` file from an untrusted source as untrusted input. Review explicit save and export destinations before execution and remove confidential data from a public reproduction. Report a suspected vulnerability privately through alexandrebarroso.com.

An evaluator change requires a manually checkable finite case, an automated regression, a declared response type and exactness boundary, and a source locator with bounded oracle when it reproduces published work. Missing candidates, weights, normalizers, transports, or observation bridges must not be filled with invented values.

PhonoScript is free and open-source software under the MIT License.

Copyright © 2026 Alexandre Menezes Barroso.